

Práctica 1

Optimización de algoritmos secuenciales

Pautas:

Para los ejercicios de matrices tendrán que descargar los archivos que las almacenan. Para esto, acceder al sitio de la facultad:

<https://sherlock.info.unlp.edu.ar:777/share/nfvWXPve>

Nota: Es normal que el navegador muestre una advertencia “*La conexión no es privada*” con recomendaciones de seguridad. Esto se debe a que la Facultad tiene un certificado expirado que está en vías de actualizarse. Mientras tanto, el sitio no representa ninguna amenaza y podrán acceder.

En el navegador Chrome ir a **Avanzado** y luego a **Acceder a ... (sitio no seguro)** En otros navegadores el proceso es similar.

Primero, resuelva y analice cada problema por su cuenta, utilizando su propio razonamiento y conocimientos. Posteriormente, utilice dos o más motores de inteligencia artificial para profundizar en el tema, comparar respuestas analizando similitudes y diferencias entre estos motores de IA.

Analice que posibilidades ofrecen otros lenguajes de programación de forma nativa, interpretados y compilados, como alternativa al lenguaje C para resolver cada problema. Asimismo, analice diferencias entre compiladores del lenguaje C, por ejemplo gcc e icc de Intel.

1. Álgebra de Matrices

- a. Analizar el programa **mm_naive.c**, que implementa la multiplicación de matrices cuadradas $C = A.B$, donde A , B y C son matrices de tamaño $N \times N$.

- i. Compilar sin optimizaciones:

```
gcc -o mm_naive mm_naive.c
```

- ii. Ejecutar la versión original para distintos tamaños de matriz (utilizando los archivos provistos por la cátedra para cada valor de N):

```
./mm_naive N AN.m BfN.m RN.m
```

Donde N es la dimensión de las matrices.

AN.m y **BfN.m** son los archivos que contienen las matrices A y B , respectivamente, almacenadas en formato row-major (ordenadas por filas consecutivas).

RN.m es el archivo que contiene la matriz resultado correcta (también en formato row-major), utilizada para validar que el cálculo de $C=A.B$ sea correcto.

- iii. Comparar el tiempo de ejecución de la versión original con una versión modificada en la que se eliminan las funciones `getValor()` y `setValor()`, accediendo directamente a las posiciones de las matrices. ¿Son realmente necesarias estas funciones? Analizar el *overhead* generado por las llamadas a función dentro de las iteraciones principales del algoritmo.
- iv. Partiendo de la versión del código en la que se eliminan las funciones `getValor()` y `setValor()`, comparar una implementación en la que las matrices A , B y C se almacenan por filas (*row-major*), con otra en la que la matriz B se almacena por columnas (*column-major*). Ejecutar la versión en la que la matriz B se almacena en memoria por columnas, para distintos tamaños de matriz, de la siguiente manera:

```
./mm_naive N AN.m BN.m RN.m
```

Donde **BN.m** es el archivo que contiene la matriz B ordenada por columnas.

Los demás parámetros se mantiene igual a la versión anterior.

Comparar los tiempos de ejecución de ambas versiones.

¿Qué puede concluirse a partir de las diferencias observadas?

¿Cómo se relacionan estos resultados con el principio de localidad?

- b. Utilizar parte del código de **mm_naive.c** para implementar una solución para la multiplicación de matrices $C=A.A$, donde A es una matriz cuadrada de tamaño $N \times N$. En el mismo código deben compararse dos estrategias:

Ambas matrices ordenadas por filas: Tanto la matriz de la izquierda como la de la derecha están almacenadas en memoria por filas (*row-major*).

Matriz derecha reordenada por columnas: La matriz de la izquierda mantiene el orden por filas. La matriz de la derecha se construye a partir de la matriz izquierda modificando su orden de almacenamiento para que quede organizada por columnas (*column-major*). El tiempo de ejecución debe incluir el tiempo necesario para reordenar la matriz.

Ejecutar para distintos tamaños de matriz de la siguiente forma:

```
./mm_aa N AN.m
```

¿Cuál de las dos estrategias obtiene mejor rendimiento?
 ¿Tiene impacto en el tiempo de ejecución el número de instrucciones adicionales para reordenar una matriz en memoria? ¿Qué influye en la diferencia de tiempos?

- c. Describir brevemente cómo funciona el algoritmo **mmb1k.c**, que realiza la multiplicación de matrices cuadradas $N \times N$ utilizando la técnica por bloques de tamaño $BS \times BS$.

Compilar sin optimizaciones:

```
gcc -o mmb1k mmb1k.c
```

Ejecutar para distintos tamaños de matriz (utilizando los archivos provistos por la cátedra para cada valor de N) variando la dimensión del bloque (BS) en potencias de 2, de la siguiente forma:

```
./mmb1k N BS AN.m BN.m RN.m
```

*Donde **BS** es la dimensión del bloque.*

Los demás parámetros se mantiene igual a las versiones anteriores.

Encontrar el tamaño de bloque con el cual se obtiene el mejor rendimiento para la arquitectura utilizada.

Comparar los tiempos de ejecución, para el tamaño de bloque óptimo, con los algoritmos anteriores.

- d. Describir brevemente cómo funciona el algoritmo **mmb1k_blas.c**, similar a **mmb1k.c** del inciso anterior pero que sigue la estrategia de la biblioteca BLAS (*Basic Linear Algebra Subprograms*).

Compilar sin optimizaciones:

```
gcc -o mmb1k_blas mmb1k_blas.c
```

Ejecutar para distintos tamaños de matriz (utilizando los archivos provistos por la cátedra para cada valor de N) variando la dimensión del bloque (BS) en potencias de 2, de la siguiente forma:

```
./mmb1k N BS AN.m BfN.m RN.m
```

*Donde **BfN.m** es el archivo que contiene la matriz B almacenada en formato row-major (ordenada por filas consecutivas).*

Los demás parámetros se mantiene igual a las versiones anteriores.

- e. Implementar las soluciones para la multiplicación de matrices MU , ML , UM y LM . Donde M es una matriz de $N \times N$. L y U son matrices triangulares inferior y superior, respectivamente. Analizar los tiempos de ejecución para la solución que almacena los ceros de las triangulares respecto a la que no los almacena.

2. El programa **fib.c** calcula el término N de la serie de Fibonacci utilizando dos estrategias diferentes: una recursiva y otra iterativa.

Compilar sin optimizaciones:

```
gcc -o fib fib.c
```

Ejecutar para valores entre 0 y 50 de la siguiente forma:

```
./fib N
```

Donde **N** es el término a calcular.

Comparar el rendimiento de ambas estrategias a partir de los tiempos de ejecución obtenidos. ¿Cuál es más rápido? ¿Por qué se produce esa diferencia?

3. Optimización de instrucciones

- a. El algoritmo **comparaInstrucciones.c** evalúa el tiempo de ejecución de las operaciones aritméticas básicas, suma (+), resta (-), multiplicación (*) y división (/), aplicadas a los elementos ubicados en la misma posición de dos arreglos, **x** e **y**.

Compilar sin optimizaciones:

```
gcc -o comparaInstrucciones comparaInstrucciones.c
```

Ejecutar de la siguiente forma:

```
./comparaInstrucciones N R
```

Donde **N** es la dimensión de los arreglos.

R representa la cantidad de repeticiones con las que se ejecuta el cálculo.

Analizar el tiempo de ejecución de cada operación.

¿Qué operación demora mas tiempo?

¿Qué ocurre con los tiempos de ejecución si los valores de los arreglos son potencias de 2?

- b. En función del ejercicio anterior, analizar el algoritmo **div_vs_mul.c** que aplica dos operaciones distintas a cada elemento de un arreglo **x**.

Compilar sin optimizaciones:

```
gcc -o div_vs_mul div_vs_mul.c
```

Ejecutar de la siguiente forma:

```
./div_vs_mul N R
```

Donde los parámetros representan lo mismo que el inciso anterior.

- c. El algoritmo **modulo.c** compara el tiempo de ejecución de dos implementaciones distintas para calcular el resto de la división por **m**, donde **m** es una potencia de 2, aplicado a los elementos enteros de un arreglo de tamaño **N**.

Compilar sin optimizaciones:

```
gcc -o modulo modulo.c
```

Ejecutar de la siguiente forma:

```
./modulo N m
```

Donde **N** es la dimensión del arreglo y **m** es el divisor potencia de 2.

¿En que consiste la optimización con la que se obtiene el mejor tiempo de ejecución?

4. Iteraciones

- a. Analizar el algoritmo **direccionamiento.c**, que inicializa un arreglo con valores en 1 utilizando dos estrategias diferentes.

Compilar sin optimizaciones:

```
gcc -o direccionamiento direccionamiento.c
```

Ejecutar de la siguiente forma:

```
./direccionamiento N R
```

Donde **N** es la dimensión del arreglo.

R representa la cantidad de repeticiones con las que se ejecuta la inicialización.

¿Cuál de las dos estrategias es más rápida? ¿Por qué?

- b. Analizar el algoritmo **Gauss.c**, que calcula la suma de los N primeros números naturales y la compara con la fórmula de Gauss.

Compilar sin optimizaciones:

```
gcc -o Gauss Gauss.c
```

Ejecutar de la siguiente forma:

```
./Gauss N
```

Donde **N** es el número hasta el cual se quiere calcular la suma de los números naturales.

¿Por qué la suma es correcta para $N=2147483647$, pero incorrecta para $N=2147483648$? ¿Cómo podría solucionarlo?

5. El algoritmo **overheadIF.c** resuelve mediante tres estrategias el siguiente problema: dado un una arreglo **V** y una posición **P**, cuenta cuántos elementos de **V** son menores que el elemento en la posición **P**.

Compilar sin optimizaciones:

```
gcc -o overheadIF overheadIF.c
```

Ejecutar de la siguiente forma:

```
./overheadIF N P
```

Donde **N** es la dimensión del arreglo y **P** es la posición donde se encuentra el elemento para calcular los valores menores.

Analizar los tiempos obtenidos para cada solución. ¿Por qué existe diferencia de tiempos de ejecución entre las distintas estrategias? Evaluar las posibles fuentes de *overhead*.

6. El algoritmo **precision.c** calcula el número de Fibonacci para cada elemento de un arreglo de tamaño N .

El algoritmo compara el resultado de calcular Fibonacci de forma iterativa utilizando datos de tipo entero:

$$\begin{aligned}f_0 &= 0 \\f_1 &= 1 \\f_n &= f_{n-1} + f_{n-2}\end{aligned}$$

con el cálculo basado en el número áureo utilizando datos de coma flotante:

$$\begin{aligned}\varphi &= \frac{1+\sqrt{5}}{2} \\f_n &= \frac{\varphi^n - (1-\varphi^n)}{\sqrt{5}}\end{aligned}$$

, en precisión simple (tipo float) y doble (tipo double).

Compilar para simple precisión (tipo de datos float) con optimizaciones:

```
gcc -O2 -lm -o singlep precision.c
```

Ejecutar de la siguiente forma:

```
./singlep N
```

Donde **N** es la dimensión del arreglo.

Compilar para doble precisión (tipo de datos double) con optimizaciones:

```
gcc -O2 -DDOUBLE -lm -o doblep precision.c
```

Ejecutar de la siguiente forma:

```
./doblep N
```

Donde **N** es al dimensión del arreglo.

Analizar los tiempos de ejecución obtenidos para cada tipo de datos. ¿Qué conclusiones se pueden obtener a partir de los tiempos de ejecución y la precisión en los tipos de dato de coma flotante?

7. El algoritmo **nreinas.c** resuelve secuencialmente el problema de las N reinas, basado en *N-Queens Solutions*, de **Takaken**. Estudiar el problema y su solución.

Compilar sin optimizaciones:

```
gcc -o nreinas nreinas.c
```

Ejecutar de la siguiente forma:

```
./nreinas N
```

Donde **N** es el número de reinas.

Estudiar cómo cambia el tiempo de ejecución a medida que aumenta N . Realizar las pruebas para valores de N desde 4 hasta 20, incrementando de uno en uno. ¿Cuál es el orden asintótico de ejecución ?

8. Los algoritmos **mergeSimpleMalloc.c** y **mergeMultiplesMalloc.c** resuelven la ordenación de un arreglo de tamaño N por el método de ordenación por mezcla.

Compilar sin optimizaciones:

```
gcc -o mergeSimpleMalloc mergeSimpleMalloc.c
```

```
gcc -o mergeMultiplesMalloc mergeMultiplesMalloc.c
```

Ejecutar de la siguiente forma:

```
./mergeSimpleMalloc N
```

```
./mergeMultiplesMalloc N
```

Donde **N** es el tamaño del arreglo.

Analizar las diferencias entre ambos algoritmos. Ejecutar para distintos tamaños de N y analizar las diferencias en el tiempo de ejecución. ¿A qué se deben las diferencias en el tiempo obtenido?